

# **SIMATS ENGINEERING**

## **ASSESSMENT TOOL 2 - Medium (LABORATORY PERFORMANCE)**

**COURSE CODE: CSA09**

**COURSE NAME: Programming in  
Java**

### **COURSE OUTCOME COVERED**

**CO4:** *Design and develop GUI-based user interfaces using Abstract Window Toolkit, Swing, and Applets by implementing event handling, layout managers, AWT controls, and menus. (BL3,BL5)*

Assessment Tool Weightage: 10%

**QUESTIONS: 1**

**Total Marks: 50**

### **ANNEXURE A**

#### **LABORATORY PERFORMANCE**

##### ***Code of Conduct:***

*I p.k.salmiksha(192511318) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

##### ***Signature of the student:***

p.k.salmiksha

## ANNEXURE

### LABORATORY PERFORMANCE

#### 1. Student Registration Applet

Develop a Java Applet for a university student registration form. The applet should display labels and input controls for Student Name, Register Number, Department, Gender, and Email ID. The application should use appropriate **AWT controls** such as Label, TextField, Checkbox, CheckboxGroup, and Button. Use a suitable **layout manager** to organize the controls. When the student clicks the **Submit** button, the entered information should be displayed within the applet window.

**The student must demonstrate:**

1. Applet initialization and architecture.
2. AWT controls.
3. Layout management.
4. Button event handling.
5. Appropriate event listener interface.
6. Display of user-entered information.

#### 2. Simple Calculator Applet

Develop an Applet-based calculator that accepts two numbers and provides buttons for **Addition, Subtraction, Multiplication, and Division**.

The application should use TextField, Label, and Button controls. Appropriate event listeners must be implemented so that clicking each button performs the corresponding operation.

The program must handle invalid numeric input and division by zero appropriately.

**The student must demonstrate:**

1. Basic Applet structure.
2. AWT controls.
3. Event sources and event objects.
4. ActionListener.
5. Exception handling.
6. Display methods.

#### 3. Course Selection Applet

Develop a university course selection Applet where students can select:

- Department
- Semester
- Course
- Elective type

Use Choice, List, Checkbox, and Button controls wherever appropriate.

When the student clicks **Register**, display the selected courses and other information.

Use appropriate layout managers and event listeners.

#### 4. Traffic Signal Control Applet

Develop an AWT Applet that simulates a traffic signal with three options:

- Red
- Yellow
- Green

Use appropriate controls to allow the user to select a signal. When the user selects a signal, display the corresponding message such as **STOP**, **WAIT**, or **GO**.

Use appropriate event sources, event classes, and listener interfaces.

#### RUBRICS FOR CALCULATION:

| Criteria                                  | Weightage | Level 4 – Exemplary                                              | Level 3 – Proficient                  | Level 2 – Developing                   | Level 1 – Beginning                    |
|-------------------------------------------|-----------|------------------------------------------------------------------|---------------------------------------|----------------------------------------|----------------------------------------|
| Problem Understanding & Program Design    | 20%       | Clearly understands the problem and develops an effective design | Good understanding and logical design | Basic understanding with some gaps     | Poor understanding and unclear design  |
| Java Implementation & Concept Application | 30%       | Applies Java concepts correctly and effectively                  | Applies concepts with minor errors    | Limited application of concepts        | Incorrect or incomplete implementation |
| Program Execution & Output                | 20%       | Program executes correctly and produces accurate output          | Executes with minor errors            | Partially executes with some errors    | Fails to execute correctly             |
| Testing & Debugging                       | 15%       | Thoroughly tests and resolves errors independently               | Tests and resolves most errors        | Performs limited testing and debugging | Unable to test or debug effectively    |

|                                             |            |                                                            |                                          |                                          |                                           |
|---------------------------------------------|------------|------------------------------------------------------------|------------------------------------------|------------------------------------------|-------------------------------------------|
| <b>Documentation<br/>&amp; Presentation</b> | <b>15%</b> | Complete,<br>clear, and<br>well-organized<br>documentation | Good<br>documentation<br>with minor gaps | Basic<br>documentation with<br>some gaps | Incomplete or<br>unclear<br>documentation |
|---------------------------------------------|------------|------------------------------------------------------------|------------------------------------------|------------------------------------------|-------------------------------------------|

# 1.Student registration applet

## Source code:

```
import java.applet.Applet;
import java.awt.*;
import java.awt.event.*;

public class StudentRegistrationApplet extends Applet implements ActionListener {

    // AWT Controls
    Label titleLabel;
    Label nameLabel;
    Label regLabel;
    Label deptLabel;
    Label genderLabel;
    Label emailLabel;

    TextField nameText;
    TextField regText;
    TextField emailText;

    Choice deptChoice;

    Checkbox male;
    Checkbox female;
    Checkbox other;
    CheckboxGroup genderGroup;

    Button submitButton;

    TextArea outputArea;

    // Applet Initialization
    public void init() {

        // -----
        // 1. APPLET INITIALIZATION AND ARCHITECTURE
        // -----
        setLayout(new BorderLayout());

        // Title
        titleLabel = new Label(
            "UNIVERSITY STUDENT REGISTRATION FORM",
            Label.CENTER
        );

        add(titleLabel, BorderLayout.NORTH);

        // -----
```

```
// 2. AWT CONTROLS
```

```
// -----
```

```
nameLabel = new Label("Student Name:");
regLabel = new Label("Register Number:");
deptLabel = new Label("Department:");
genderLabel = new Label("Gender:");
emailLabel = new Label("Email ID:");
```

```
nameText = new TextField(20);
regText = new TextField(20);
emailText = new TextField(20);
```

```
// Department Choice
```

```
deptChoice = new Choice();
deptChoice.add("Computer Science and Engineering");
deptChoice.add("Information Technology");
deptChoice.add("Electronics and Communication");
deptChoice.add("Electrical and Electronics");
deptChoice.add("Mechanical Engineering");
```

```
// Gender CheckboxGroup
```

```
genderGroup = new CheckboxGroup();
```

```
male = new Checkbox("Male", genderGroup, false);
female = new Checkbox("Female", genderGroup, false);
other = new Checkbox("Other", genderGroup, false);
```

```
// Submit Button
```

```
submitButton = new Button("Submit");
```

```
// -----
```

```
// 3. LAYOUT MANAGEMENT
```

```
// -----
```

```
Panel formPanel = new Panel();
```

```
formPanel.setLayout(new GridLayout(6, 2, 10, 10));
```

```
formPanel.add(nameLabel);
formPanel.add(nameText);
```

```
formPanel.add(regLabel);
formPanel.add(regText);
```

```
formPanel.add(deptLabel);
formPanel.add(deptChoice);
```

```
formPanel.add(genderLabel);
```

```
Panel genderPanel = new Panel();
genderPanel.setLayout(new FlowLayout(FlowLayout.LEFT));
```

```

genderPanel.add(male);
genderPanel.add(female);
genderPanel.add(other);

formPanel.add(genderPanel);

formPanel.add(emailLabel);
formPanel.add(emailText);

formPanel.add(new Label(""));
formPanel.add(submitButton);

add(formPanel, BorderLayout.CENTER);

// -----
// 4. BUTTON EVENT HANDLING
// -----

submitButton.addActionListener(this);

// -----
// 6. DISPLAY OF USER-ENTERED INFORMATION
// -----

outputArea = new TextArea(
    "Submitted information will appear here.",
    8,
    50,
    TextArea.SCROLLBARS_VERTICAL_ONLY
);

outputArea.setEditable(false);

add(outputArea, BorderLayout.SOUTH);
}

// -----
// 5. APPROPRIATE EVENT LISTENER INTERFACE
// -----

public void actionPerformed(ActionEvent e) {

    if (e.getSource() == submitButton) {

        String name = nameText.getText();
        String regNo = regText.getText();
        String department = deptChoice.getSelectedItem();
        String email = emailText.getText();

        Checkbox selectedGender = genderGroup.getSelectedCheckbox();

```

```

String gender;

if (selectedGender != null) {
    gender = selectedGender.getLabel();
} else {
    gender = "Not Selected";
}

// -----
// 6. DISPLAY USER-ENTERED INFORMATION
// -----

outputArea.setText(
    "STUDENT REGISTRATION DETAILS\n\n" +
    "Student Name   : " + name + "\n" +
    "Register Number : " + regNo + "\n" +
    "Department      : " + department + "\n" +
    "Gender          : " + gender + "\n" +
    "Email ID        : " + email
);
}
}
}

```

Output:

Applet Viewer: StudentRegistrationApplet.class

Applet

UNIVERSITY STUDENT REGISTRATION FORM

Student Name: salmiksha

Register Number: 192511318

Department: Computer Science and Engineering

Gender: ☒ Male ☐ Female ☐ Other

Email ID: 192511452.simats@saveetha.com

Submit

Submitted information will appear here.

Applet started.

Applet Viewer: StudentRegistrationApplet.class

Applet

UNIVERSITY STUDENT REGISTRATION FORM

Student Name: salmiksha

Register Number: 192511318

Department: Computer Science and Engineering

Gender: ☒ Male ☐ Female ☐ Other

Email ID: 192511318.simats@saveetha.com

Submit

STUDENT REGISTRATION DETAILS

Student Name : salmiksha  
 Register Number : 192511318  
 Department : Computer Science and Engineering  
 Gender : Male  
 Email ID : 192511318.simats@saveetha.com

Applet started.



## 2.Simple calculator applet

### Source code:

```
import java.applet.Applet;
import java.awt.*;
import java.awt.event.*;

public class SimpleCalculatorApplet extends Applet implements ActionListener {

    // AWT Controls
    Label titleLabel;
    Label num1Label;
    Label num2Label;
    Label resultLabel;

    TextField num1Text;
    TextField num2Text;
    TextField resultText;

    Button addButton;
    Button subButton;
    Button mulButton;
    Button divButton;

    // -----
    // 1. BASIC APPLET STRUCTURE
    // -----
    public void init() {

        setLayout(new GridLayout(6, 2, 10, 10));

        // -----
        // 2. AWT CONTROLS
        // -----

        titleLabel = new Label("SIMPLE CALCULATOR", Label.CENTER);

        num1Label = new Label("Enter First Number:");
        num2Label = new Label("Enter Second Number:");
        resultLabel = new Label("Result:");

        num1Text = new TextField();
        num2Text = new TextField();
        resultText = new TextField();

        resultText.setEditable(false);

        addButton = new Button("Addition");
        subButton = new Button("Subtraction");
```

```

mulButton = new Button("Multiplication");
divButton = new Button("Division");

// Add controls to applet

add(titleLabel);
add(new Label(""));

add(num1Label);
add(num1Text);

add(num2Label);
add(num2Text);

add(addButton);
add(subButton);

add(mulButton);
add(divButton);

add(resultLabel);
add(resultText);

// -----
// 4. ACTIONLISTENER
// -----

addButton.addActionListener(this);
subButton.addActionListener(this);
mulButton.addActionListener(this);
divButton.addActionListener(this);
}

// -----
// 3. EVENTS AND EVENT OBJECTS
// -----
public void actionPerformed(ActionEvent e) {

// -----
// 5. EXCEPTION HANDLING
// -----

try {

    double num1 = Double.parseDouble(num1Text.getText());
    double num2 = Double.parseDouble(num2Text.getText());

    double result = 0;

    if (e.getSource() == addButton) {

```

```

        result = num1 + num2;

    }
    else if (e.getSource() == subButton) {

        result = num1 - num2;

    }
    else if (e.getSource() == mulButton) {

        result = num1 * num2;

    }
    else if (e.getSource() == divButton) {

        if (num2 == 0) {
            resultText.setText("Cannot divide by zero");
            return;
        }

        result = num1 / num2;
    }

    // -----
    // 6. DISPLAY METHODS
    // -----

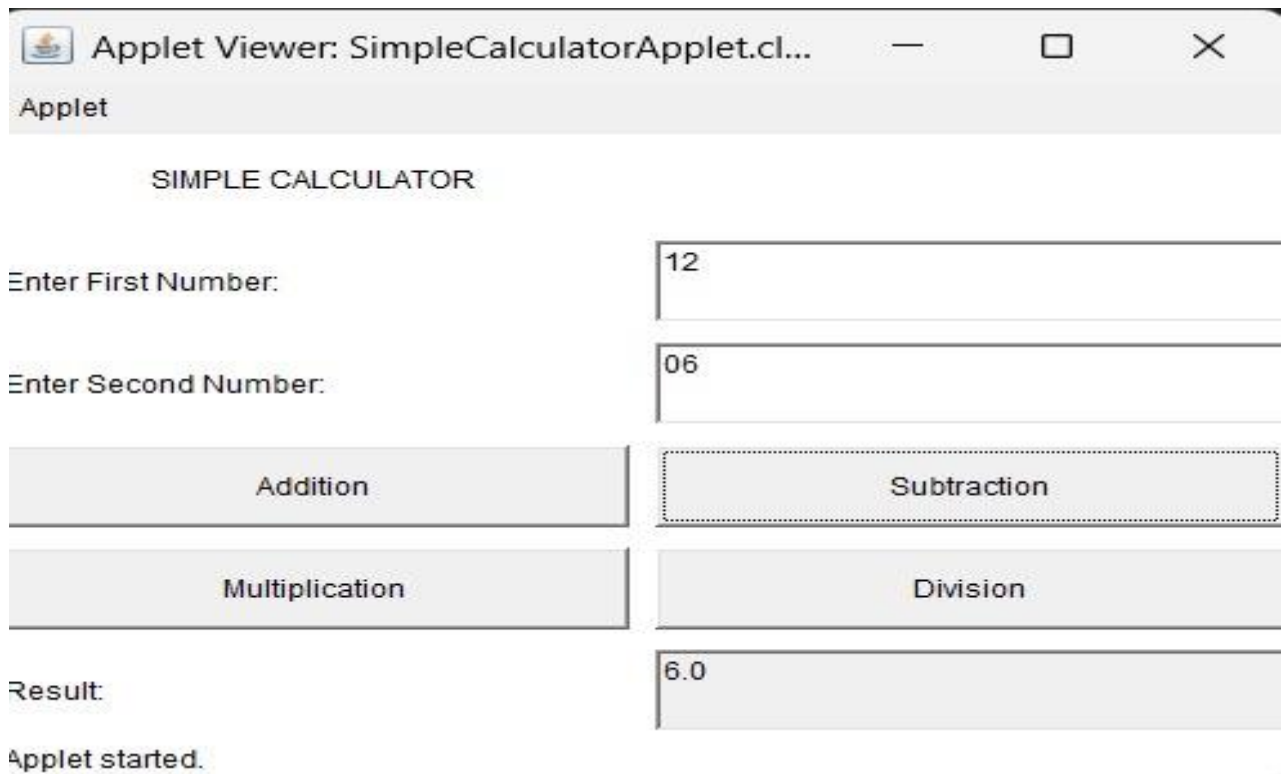
    resultText.setText(String.valueOf(result));

}
catch (NumberFormatException ex) {

    resultText.setText("Invalid numeric input");
}
}
}

```

Output:



### 3.Course selection applet

#### Source code:

```
import java.applet.Applet;
import java.awt.*;
import java.awt.event.*;

public class CourseSelectionApplet extends Applet implements ActionListener {

    // AWT Controls
    Label titleLabel;
    Label departmentLabel;
    Label semesterLabel;
    Label courseLabel;
    Label electiveLabel;

    Choice departmentChoice;
    Choice semesterChoice;

    List courseList;

    Checkbox technicalCheckbox;
    Checkbox openElectiveCheckbox;

    Button registerButton;
```

TextArea outputArea;

```
// -----  
// 1. BASIC APPLET STRUCTURE  
// -----  
public void init() {  
  
    setLayout(new BorderLayout(10, 10));  
  
    // -----  
    // 2. AWT CONTROLS  
    // -----  
  
    titleLabel = new Label(  
        "UNIVERSITY COURSE SELECTION",  
        Label.CENTER  
    );  
  
    departmentLabel = new Label("Department:");  
    semesterLabel = new Label("Semester:");  
    courseLabel = new Label("Select Course:");  
    electiveLabel = new Label("Elective Type:");  
  
    // Department Choice  
    departmentChoice = new Choice();  
    departmentChoice.add("Computer Science and Engineering");  
    departmentChoice.add("Information Technology");  
    departmentChoice.add("Electronics and Communication");  
    departmentChoice.add("Electrical and Electronics");  
    departmentChoice.add("Mechanical Engineering");  
  
    // Semester Choice  
    semesterChoice = new Choice();  
    semesterChoice.add("Semester 1");  
    semesterChoice.add("Semester 2");  
    semesterChoice.add("Semester 3");  
    semesterChoice.add("Semester 4");  
    semesterChoice.add("Semester 5");  
    semesterChoice.add("Semester 6");  
  
    // Course List  
    courseList = new List(5, false);  
    courseList.add("Data Structures");  
    courseList.add("Operating Systems");  
    courseList.add("Database Management Systems");  
    courseList.add("Computer Networks");  
    courseList.add("Java Programming");  
  
    // Elective Checkboxes  
    technicalCheckbox = new Checkbox("Technical Elective");  
    openElectiveCheckbox = new Checkbox("Open Elective");  
  
    // Register Button  
    registerButton = new Button("Register");
```

```

// Output Area
outputArea = new TextArea(8, 50);
outputArea.setEditable(false);

// -----
// 3. LAYOUT MANAGEMENT
// -----

Panel formPanel = new Panel();

formPanel.setLayout(new GridLayout(5, 2, 10, 10));

formPanel.add(departmentLabel);
formPanel.add(departmentChoice);

formPanel.add(semesterLabel);
formPanel.add(semesterChoice);

formPanel.add(courseLabel);
formPanel.add(courseList);

formPanel.add(electiveLabel);

Panel electivePanel = new Panel();
electivePanel.setLayout(
    new FlowLayout(FlowLayout.LEFT)
);

electivePanel.add(technicalCheckbox);
electivePanel.add(openElectiveCheckbox);

formPanel.add(electivePanel);

formPanel.add(new Label(""));
formPanel.add(registerButton);

add(titleLabel, BorderLayout.NORTH);
add(formPanel, BorderLayout.CENTER);
add(outputArea, BorderLayout.SOUTH);

// -----
// 4. EVENT LISTENER
// -----

registerButton.addActionListener(this);
}

// -----
// 5. BUTTON EVENT HANDLING

```

```

// -----

public void actionPerformed(ActionEvent e) {

    if (e.getSource() == registerButton) {

        String department =
            departmentChoice.getSelectedItemAt();

        String semester =
            semesterChoice.getSelectedItemAt();

        String course =
            courseList.getSelectedItemAt();

        String elective = "";

        if (technicalCheckbox.getState()) {
            elective = "Technical Elective";
        }

        if (openElectiveCheckbox.getState()) {

            if (!elective.equals("")) {
                elective = elective + ", Open Elective";
            } else {
                elective = "Open Elective";
            }
        }

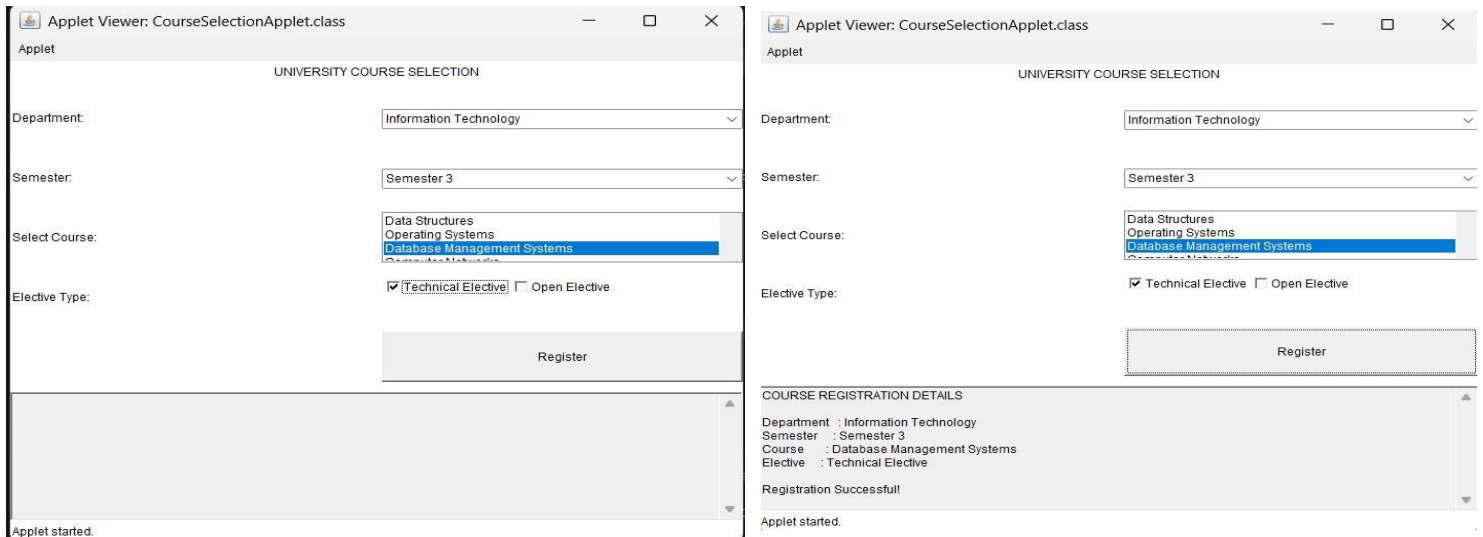
        if (elective.equals("")) {
            elective = "Not Selected";
        }

        // -----
        // 6. DISPLAY SELECTED INFORMATION
        // -----

        outputArea.setText(
            "COURSE REGISTRATION DETAILS\n\n" +
            "Department   : " + department + "\n" +
            "Semester      : " + semester + "\n" +
            "Course        : " + course + "\n" +
            "Elective       : " + elective + "\n\n" +
            "Registration Successful!"
        );
    }
}
}

```

## Output:



## 4. Traffic signal control applet

### Source code:

```
import java.applet.Applet;
import java.awt.*;
import java.awt.event.*;

public class TrafficSignalApplet extends Applet implements ItemListener {

    Label titleLabel;
    Label messageLabel;

    Checkbox redButton;
    Checkbox yellowButton;
    Checkbox greenButton;

    CheckboxGroup signalGroup;

    // -----
    // 1. BASIC APPLET STRUCTURE
    // -----
    public void init() {

        setLayout(new BorderLayout(10, 10));

        // -----
```



```
// 2. AWT CONTROLS
```

```
// -----
```

```
titleLabel = new Label(  
    "TRAFFIC SIGNAL CONTROL",  
    Label.CENTER  
);
```

```
messageLabel = new Label(  
    "Select a traffic signal",  
    Label.CENTER  
);
```

```
// CheckboxGroup allows only one signal  
// to be selected at a time.
```

```
signalGroup = new CheckboxGroup();
```

```
redButton = new Checkbox(  
    "Red",  
    signalGroup,  
    false  
);
```

```
yellowButton = new Checkbox(  
    "Yellow",  
    signalGroup,  
    false  
);
```

```
greenButton = new Checkbox(  
    "Green",  
    signalGroup,  
    false  
);
```

```
// -----
```

```
// 3. LAYOUT MANAGEMENT
```

```
// -----
```

```
Panel signalPanel = new Panel();
```

```
signalPanel.setLayout(  
    new FlowLayout(FlowLayout.CENTER, 20, 20)  
);
```

```
signalPanel.add(redButton);  
signalPanel.add(yellowButton);  
signalPanel.add(greenButton);
```

```
add(titleLabel, BorderLayout.NORTH);
```

```

add(signalPanel, BorderLayout.CENTER);
add(messageLabel, BorderLayout.SOUTH);

// -----
// 4. EVENT LISTENER
// -----

redButton.addItemListener(this);
yellowButton.addItemListener(this);
greenButton.addItemListener(this);
}

// -----
// 5. EVENT HANDLING
// -----

public void itemStateChanged(ItemEvent e) {

    // -----
    // 6. EVENT SOURCE AND EVENT OBJECT
    // -----

    if (e.getSource() == redButton) {

        messageLabel.setText("STOP");

    }
    else if (e.getSource() == yellowButton) {

        messageLabel.setText("WAIT");

    }
    else if (e.getSource() == greenButton) {

        messageLabel.setText("GO");

    }
}
}

```

# Output:

